
PinnDE: Physics-Informed Neural Networks for Solving Differential Equations

Jason Matthews

Department of Mathematics and Statistics
Memorial University of Newfoundland
St. John's, NL, A1C 5S7, Canada
jbmattthews@mun.ca

Alex Bihlo

Department of Mathematics and Statistics
Memorial University of Newfoundland
St. John's, NL, A1C 5S7, Canada
abihlo@mun.ca

Keywords: PinnDE, differential equations, physics-informed neural networks, deep operator networks

Abstract

In recent years the study of deep learning for solving differential equations has grown substantially. The use of physics-informed neural networks (PINNs) and deep operator networks (DeepONets) have emerged as two of the most useful approaches in approximating differential equation solutions using machine learning. Here, we introduce PinnDE, an open-source Python library for solving differential equations with both PINNs and DeepONets. We give a brief review of both PINNs and DeepONets, introduce PinnDE along with the structure and usage of the package, and present worked examples to show PinnDE's effectiveness in approximating solutions of systems of differential equations with both PINNs and DeepONets.

1 Introduction

Powerful numerical algorithms for solving differential equations have been developed for well over one hundred years [34]. Modern deep learning has been applied and studied through many diverse fields [12, 39], however the use of deep learning for solving differential equations is a relatively new field. While the first ideas of these methods were proposed in [20, 21], in [32] the notion of a physics-informed neural network (PINN) was popularized. Compared to finite difference methods, finite element methods or finite volume methods that use numerical differentiation for computing derivatives, using automatic differentiation [3] in deep learning-based approaches provides a meshless alternative to solving differential equations. While [32] first proposed this for solving forward and inverse problems, many other variations of differential equations have been demonstrated to be able to be solved by PINNs. PINNs can not only be used for solving differential equations directly, the so-called *forward problem*, they can also be used for parameter identification or learning of differential equations itself from data, which is referred to as the *inverse problem*, as discussed in [26, 33, 45]. Solving fractional differential equations has been proposed and improved upon in the works of [30, 36]. Integro-differential equations have been shown to be amenable to PINN-based methods [46] as well. Stochastic differential equations were tackled with only slight variations to standard PINN architecture [15, 44, 49]. There has also been some initial work indicating that including geometric properties of differential equations can improve the numerical solutions obtainable using physics-informed machine learning, cf. [2, 10].

While there has been much research done on the different variations of differential equations solvable by PINNs, work on improving the ability of PINNs in general has been a large focus over the past several years as well. Adaptive methods for collocation point sampling have been a common field of study, where residual-based adaptive refinement (RAR) is a popular method which has been proposed

and improved upon in the works of [17, 24, 29, 43, 47]. More specific adaptive collocation point have been used in [37, 42]. Techniques for providing adaptive scaling parameters on the PINN architecture are developed in [41] and [27]. Meta learning, a method for a neural network learned optimizer to optimize a PINN as opposed to a hand-crafted optimizer, has been a relatively new area of research to improve physics-informed machine learning for solving differential equations [4, 23, 31].

While PINNs are the main focus using deep learning for solving differential equations, a second, more general approach has grown in popularity as a tool for solving differential equations as well. Deep operator networks (DeepONets), first proposed in [24], provide a general method for solving differential equations. A key distinction separates PINNs from DeepONets. While a PINN is trained to approximate the solution function, $u(x)$, for a single instance of a differential equation, a DeepONet is designed to learn the underlying solution operator, G . The goal is to approximate the mapping from an input function f (such as the initial condition, boundary conditions, or forcing term) to the corresponding solution function, such that $u(x) \approx G(f)(x)$. This operator learning framework provides a significant advantage: Once trained, the DeepONet can be evaluated on new, unseen input functions without requiring retraining, making it highly efficient for solving parametric problems. Furthermore, the learned operator can be applied sequentially for long time domain predictions. By using the predicted solution at one time step as the new initial condition for the subsequent step, the network can perform time-stepping simulations, as demonstrated by [40]. Many different DeepONet variations have been proposed throughout [16, 22, 35, 38, 48] which show progress in the improvement in this architecture.

The Python machine learning frameworks TensorFlow, PyTorch, and JAX [1, 7, 14] are commonly used for implementing deep learning algorithms and architecture. Despite this and deep learning for differential equations becoming popular research fields, there exist few software packages which implement these ideas using low-code solutions. While existing packages like DeepXDE [25] provide powerful and flexible frameworks, they often demand significant software development experience. This results in researchers and educators having to re-create already well-understood implementations from scratch. Here, we introduce PinnDE, an open-source Python library for solving differential equations with PINNs and DeepONets implemented in TensorFlow and JAX. PinnDE provides a user-friendly interface in which only a minimal amount of functions is given to a user. PinnDE also provides simple to read implementations which we believe gives this package the ability to be used in education and research effectively, requiring minimal background knowledge on physics-informed machine learning by its users.

This paper is organized as follows: We briefly discuss the theory behind PINNs and DeepONets in Section 2, and present a short but self-contained general overview of their implementation. Next, in Section 3 we introduce PinnDE, providing a summary of the structure of the package with explanations on its usage. Then, in Section 4 we provide some worked examples of commonly solved ordinary and partial differential equations using different methods to show the effectiveness of PinnDE. Finally, Section 5 contains the conclusions and some discussions about further plans for developing PinnDE.

2 Background

In this section we give a short review of a physics-informed neural network (PINN), deep operator network (DeepONet), and inverse PINNs which are the basis of the algorithms and models implemented in PinnDE. We then give an overview of current adaptive point sampling strategies and optimization routines which PinnDE covers.

2.1 Physics-informed neural networks for forward problem

Physics-informed neural networks (PINNs) were first introduced in [20] and were later popularized when re-introduced in [32]. The general idea consists of taking a deep neural network as a surrogate approximation for the solution of a system of differential equations. This has the advantage that derivatives of the neural network approximation to the solution of the system of differential equations can be computed with automatic differentiation [3] rather than relying on numerical differentiation as is typically the case for standard numerical methods such as finite difference, finite volume or finite element methods. As a consequence, physics-informed neural networks are truly meshless methods as the derivative computations can be done in single points, without relying on the introduction

of a computational mesh as is the case for most standard numerical methods. The neural network surrogate solution is being obtained by solving an optimization problem that involves fitting the weights and biases of the network. This is done such that at a collection of finitely many (typically randomly chosen) collocation points a loss function combining the differential equation along with any supplied initial and/or boundary conditions is minimized. We make this general procedure more precise in the following.

We consider a general initial-boundary value problem for a system of L partial differential equations over a spatio-temporal domain $[t_0, t_f] \times \Omega$, where $\Omega \subset \mathbb{R}^d$, $t_0 \in \mathbb{R}$ denotes the initial integration time, and $t_f \in \mathbb{R}$ denotes the final integration time, given by

$$\Delta^l(t, x, u_{(n)}) = 0, \quad l = 1, \dots, L, \quad t \in [t_0, t_f], \quad x \in \Omega, \quad (1a)$$

$$l^i(x, u_{(n_i)}|_{t=t_0}) = 0, \quad l_i = 1, \dots, L_i, \quad x \in \Omega, \quad (1b)$$

$$B^{l_b}(t, x, u_{(n_b)}) = 0, \quad l_b = 1, \dots, L_b, \quad t \in [t_0, t_f], \quad x \in \partial\Omega, \quad (1c)$$

where t denotes the time variable and $x = (x_1, \dots, x_d)$ denotes the tuple of spatial independent variables. The dependent variables are denoted by $u = (u^1, \dots, u^q)$, and $u_{(n)}$ denotes the tuple of all derivatives of the dependent variable with respect to both t and x up to order n . The initial conditions are represented through the initial value operator $l = (l^1, \dots, l^{L_i})$, and similarly the boundary conditions are included using the boundary value operator $B = (B^1, \dots, B^{L_b})$.

We denote a deep neural network as $\mathcal{N}^\theta$ with parameters θ , which includes all the weights and biases of all layers of the neural network. The goal of a physics-informed neural network is to learn to interpolate the global solution of the system of differential equations over $[t_0, t_f] \times \Omega$ as the parameterization $u^\theta(t, x) = \mathcal{N}^\theta(t, x)$, where $u^\theta(t, x) \approx u(t, x)$. This is done by minimizing the loss function

$$\mathcal{L}(\theta) = \mathcal{L}_\Delta(\theta) + \gamma_i \mathcal{L}_i(\theta) + \gamma_b \mathcal{L}_b(\theta), \quad (2)$$

where $\gamma_i, \gamma_b \in \mathbb{R}^+$ are weighting parameters for the individual loss contributions, and we composite the differential equation, initial value, and boundary value loss, respectively, which are given by

$$\mathcal{L}_\Delta(\theta) = \frac{1}{N_\Delta} \sum_{i=1}^{N_\Delta} \sum_{l=1}^L |\Delta^l(t_\Delta^i, x_\Delta^i, u_{(n)}^\theta(t_\Delta^i, x_\Delta^i))|^2, \quad (3a)$$

$$\mathcal{L}_i(\theta) = \frac{1}{N_i} \sum_{i=1}^{N_i} \sum_{l_i=1}^{L_i} |l^{l_i}(t_i^i, x_i^i, u_{(n_i)}^\theta(t_i^i, x_i^i))|^2, \quad (3b)$$

$$\mathcal{L}_b(\theta) = \frac{1}{N_b} \sum_{i=1}^{N_b} \sum_{l_b=1}^{L_b} |B^{l_b}(t_b^i, x_b^i, u_{(n_b)}^\theta(t_b^i, x_b^i))|^2, \quad (3c)$$

where $\mathcal{L}_\Delta(\theta)$ corresponds to the differential equation loss based on Eqn. (1a), $\mathcal{L}_i(\theta)$ is the initial value loss stemming from Eqn. (1b), and $\mathcal{L}_b(\theta)$ denotes the boundary value loss derived from Eqn. (1c). The neural network evaluates the losses over a set of collocation points, where we have $\{(t_\Delta^i, x_\Delta^i)\}_{i=1}^{N_\Delta}$ for the system, $\{(t_i^i, x_i^i)\}_{i=1}^{N_i}$ for the initial values, and $\{(t_b^i, x_b^i)\}_{i=1}^{N_b}$ for the boundary values, with N_Δ , N_i and N_b denoting the number of differential equation, initial condition and boundary condition collocation points, respectively.

This method is considered *soft-constrained*, as the network is forced to learn the initial and boundary conditions and composite their loss, which is described in [32]. A downside of this approach is that while the initial and boundary values are known exactly, they will in general not be satisfied exactly by the learned neural network owing to being enforced only through minimizing the loss function. An alternative strategy is to *hard-constrain* the network with the initial and/or boundary conditions, following what was first proposed in [20], which structures the neural network itself in a way so that the output of the network automatically satisfies the initial and/or boundary conditions. As such, only the differential equation loss then has to be minimized and the initial and boundary loss components in (2) are not required. The general idea of hard-constraining the initial and boundary conditions is hence to design a suitable ansatz for the neural network surrogate solution that makes sure that the initial and boundary conditions are exactly satisfied for all values of the trainable neural network. The precise form of the class of suitable hard constraints depends on the particular form of the initial and boundary value operators. We show some specific implementations in Section

3.1.1. A more formalized discussion can also be found in [9]. For many problems, it has been shown that hard-constraining the initial and boundary conditions improves the training performance of neural network based differential equations solvers, and leads to lower overall errors in the obtained solutions of these solvers, cf. [9]. One notable exception are cases where the solution of a differential equation is not differentiable everywhere, in which case soft constraints typically outperform hard constraints, see e.g. [8].

Above we have introduced physics-informed neural networks for systems of partial differential equations for general initial–boundary value problems. The same method can also be applied for ordinary differential equations. For ordinary differential equations, both initial value problems and boundary value problems can be considered.

We first consider a system of L ordinary differential equations over the temporal domain $[t_0, t_f]$,

$$\Delta^l(t, u_{(n)}) = 0, \quad l = 1, \dots, L, \quad t \in [t_0, t_f] \quad (4)$$

where $u_{(n)}$ are the total derivatives with respect to t , and Δ^l are $(m+1)$ -th order differential functions, with initial conditions,

$$l^i(u_{(n_i)}|_{t=t_0}) = 0, \quad l_i = 1, \dots, L_i \quad (5)$$

where the associated loss function for the physics-informed neural network is represented as the composite loss of Eqn. (3a) and Eqn. (3b), with Δ^l and l^i corresponding to Eqn. (4) and Eqn. (5). One can also consider a boundary value problem with the boundary conditions

$$B^{l_b}(t, u_{(n_b)}) = 0, \quad l_b = 1, \dots, L_b, \quad t \in \{t_0, t_f\}, \quad (6)$$

in which case the physics-informed loss function is represented as the composite loss of Eqn. (3a) and Eqn. (3c), with Δ^l and B^{l_b} corresponding to Eqn. (4) and Eqn. (6).

Hard constraining these networks follows a similar procedure as described above, with the initial/boundary values being exactly enforced for all values of the neural network surrogate solution. We refer to Section 3.1.1 for further details.

2.2 Physics-informed neural networks for inverse problems

Physics-informed neural networks for inverse problems were introduced in [32], and resemble closely the structure outlined in Section 2.1, and the description of PINN methods in the previous Section 2.1 holds true for inverse PINNs as well. The key distinction is for a deep neural network to learn to interpolate the global solution of a system of differential equations as well as parameters given in the system of differential equations. This is done by using experimental data to approximate said parameters.

In more detail, we consider the same system of partial differential equations described by Eqn. (1), with the addition of a parameter term

$$\Delta^l(t, x, u_{(n)}, \lambda) = 0, \quad l = 1, \dots, L, \quad t \in [t_0, t_f], \quad x \in \Omega, \quad (7a)$$

$$l^i(x, u_{(n_i)}, \lambda|_{t=t_0}) = 0, \quad l_i = 1, \dots, L_i, \quad x \in \Omega, \quad (7b)$$

$$B^{l_b}(t, x, u_{(n_b)}, \lambda) = 0, \quad l_b = 1, \dots, L_b, \quad t \in [t_0, t_f], \quad x \in \partial\Omega, \quad (7c)$$

where $\lambda = (\lambda_1, \dots, \lambda_p)$ denotes the tuple of parameters in this system of differential equations, with all other definitions given in Section 2.1. These parameters may also be functions of the independent variables, the dependent variables as well as the derivatives of the dependent variables.

In this case, for a neural network $\mathcal{N}^\theta$ with weights and biases θ , the goal of an inverse PINN is to learn to interpolate the global solution of the system of differential equations over $[t_0, t_f] \times \Omega$ and identify the values of the parameters of the equations as the parameterization $(u^\theta, \lambda^\theta) = (\mathcal{N}_1^\theta, \dots, \mathcal{N}_{q+p}^\theta) = \mathcal{N}^\theta(t, x)$, where $u^\theta(t, x) \approx u(t, x)$, and $\lambda^\theta \approx \lambda$.

This is done by adding a loss term for the experimental data to the loss function given by Eqn. (2). For a system of L partial derivative equations, we add the term $\mathcal{L}_d(\theta)$ given by

$$\mathcal{L}_d(\theta) = \frac{1}{N_d} \sum_{i=1}^{N_d} \sum_{k=1}^q |\mathcal{N}_k^\theta(t_d^i, x_d^i) - u^k(t_d^i, x_d^i)|^2, \quad (8)$$

where the network is evaluated over the collected data points sampled at the spatio-temporal data collocation points $\{(t_d^i, x_d^i)\}_{i=1}^{N_d}$, with N_d denoting the number of data points sampled. For each of the solution components of the neural network approximation $\mathcal{N}_k^\theta(t, x)$, $k = 1, \dots, q$, the mean squared error between this approximate solution and the experimental data of the underlying true solution $u^k(t, x)$ sampled at the data point locations is computed. In this setting, $\mathcal{N}^\theta$ learns to not only obtain a particular solution to a parameterized differential equation but to also fit the parameters λ to the experimental data samples.

2.3 Deep Operator Networks

Deep operator networks (DeepONets) were first introduced in [24] based on theoretical results of [11]. This idea replaces learning of a particular solution of the system of differential equations with learning the solution operator acting on a specific initial–boundary condition for the system of differential equations (1). That is, let $G(u_0)(t, x)$ be a solution operator for (1) acting on the particular initial condition u_0 yielding the solution of the initial–boundary value at the spatio-temporal point (t, x) , i.e. $u(t, x) = G(u_0)(t, x)$. That is, in comparison to standard physics-informed neural networks, deep operator networks require two inputs, the spatio-temporal point where the solution should be evaluated (this is the same input as for physics-informed neural networks), and the particular initial (or boundary) condition to which the solution operator should be applied. Since the initial condition is a function, it first has to be sampled on a finite dimensional subspace to be included into the neural network. The most prominent way to accomplish this is to sample a finite collection of N_s *sensor points* $\{x_i\}_{i=1}^{N_s}$, $x_i \in \Omega$ and evaluate the initial condition at those sensor points, yielding the tuple $(u_0(x_1), \dots, u_0(x_{N_s}))$ that is used as an input for the deep operator network. A similar strategy is followed for pure boundary value problems. A deep operator network then uses two separate networks to combine the sampled initial conditions and independent variables. The *branch network* processes the initial condition sampled values, and the *trunk network* processes the independent variables. In the vanilla deep operator network the output vectors of the two sub-networks are then combined via a dot product. The output of deep operator network can be expressed as

$$\mathcal{G}^\theta(u_0(x))(t, x) = \mathcal{B}^\theta(u_0(x_1), \dots, u_0(x_{N_s})) \cdot \mathcal{T}^\theta(t, x)$$

With $\mathcal{B}^\theta = (\mathcal{B}_1^\theta, \dots, \mathcal{B}_p^\theta)$ representing the output of the branch net and $\mathcal{T}^\theta = (\mathcal{T}_1^\theta, \dots, \mathcal{T}_p^\theta)$ representing the output of the trunk net, and p being a hyper-parameter. The loss function associated with these networks is the same as (2) when soft-constrained. Analogously to physics-informed neural networks, also deep operator networks can be hard-constrained with either standard PINN hard constraining methods, or further modified methods designed for deep operator networks, cf. [9]. A trained deep operator network thus is a solution interpolant for the solution operator, i.e. $\mathcal{G}^\theta \approx G$.

These networks come with multiple main benefits. Firstly, these networks can easily replace one initial (and/or boundary) condition with a new initial (and/or boundary) condition and do not require re-training. Secondly, as presented in [40], since these networks learn the solution operator corresponding to an initial condition, they can readily be used for time-stepping for autonomous systems of differential equations and thus extend to long time intervals and maintain the ability to further predict the solution, unlike a standard PINN, which typically fails if $t_f \gg t_0$. A downside of DeepONets is that they are typically harder to train as PINNs, since not only a single particular solution has to be interpolated, but the entire solution operator. This also places additional burdens on the computational hardware required, making DeepONets more expensive to train than PINNs.

2.4 Adaptive point sampling

The loss function described by Eqn. (2) relies on sampled collocation points across the domain of the system of PDEs. This is most commonly done using a random or quasi-random sample of the domain. One of the most frequently used sampling strategies is Latin hypercube sampling (LHS) [28] which is the sampling strategy used in the first paper re-popularizing PINNs [32].

Random sampling methods such as LHS sample collocation points independently of the solution or domain. While these methods are shown to be effective, methods of sampling collocation points which take specific solution or domain features into consideration when sampling are known as adaptive point sampling strategies. This covers a wide spectrum of strategies. Adaptive sampling strategies may include sampling strategies which add collocation points during training [25], resample

the entire collocation point distribution during training [43], or moving certain collocation points throughout the training process [19].

Frequently used adaptive sampling strategies in PINNs are residual-based adaptive strategies. These strategies use the PDE residual of collocation points as the factor which informs the sampling strategy. We give a brief overview of three of these strategies, where more details can be found in [25, 43].

Residual-based adaptive refinement (RAR) [25] is the first of these strategies. The goal of RAR is to improve the original collocation point distribution by adding points to areas of large PDE residual. After training for i epochs, a new sample of collocation points is generated. Then the PDE residual of the new points are calculated, with the n points with the largest residual added to the original distribution. Here, i, n are some predetermined number of iterations and collocation points, respectively.

Residual-based adaptive distribution (RAD) [43] is a strategy in which the entire collocation point distribution is resampled every i epochs. This is done by first sampling a larger collocation point sample than the original distribution randomly. Then from this distribution we sample points to the same size of the original distribution by the probability density function

$$p(x) \propto \frac{\epsilon(x)}{\mathbb{E}[\epsilon(x)]} + c \quad (9)$$

which is computed for every collocation point x . Here, $\epsilon(x)$ refers to the PDE residual, where $\epsilon(x) = |\Delta(x; u_{(n)})|$. Hyperparameters k, c are chosen based the desired weighting of the probability distribution function to the collocation points with higher PDE residuals.

Residual-based adaptive refinement with distribution (RAR-D) [43] is based on combining these two approaches. In this, after every i epochs of training, we add n new collocation points to the original distribution, similar to RAR. However, we add points not based on the PDE residual, but determined by the probability density function described by Eqn. (9).

3 Physics-informed neural networks for differential equations (PinnDE)

Physics-informed neural networks for differential equations (PinnDE) is an open-source library in Python providing the ability to solve differential equations with both physics-informed neural networks (PINNs) and deep operator networks (DeepONets). PinnDE is built using TensorFlow [1] and JAX [7], serving as the backend, two popular software packages for deep learning. Functionality to provide multiple backend and other deep learning frameworks such as PyTorch for use with PinnDE is in development and will be added in a future version.

The goal of PinnDE is to provide a user-friendly interface for solving systems of differential equations which can be easily shared with collaborators even if they do not have experience working with machine learning in Python. Many alternative packages for solving differential equations with PINNs or DeepONets, such as DeepXDE [25] and PINA [13], are powerful packages in this field. They provide a high level of customization and control in how a specific problem is being solved. This naturally leads to a higher amount of software needing to be written and understood by a user, which when sharing with collaborators can lead to difficulties if not everyone understands the package to the same degree. PinnDE is a tool in which we aim to bridge this gap between functionality and ease-of-use, stripping away complexity from the code resulting in an easy to read interface non-specialists can understand. This package has users create small, modular objects which represent components of a differential equation problem, which then requires just minimal interfacing with the package albeit still being able to have access to a wide range of applications, without the need to do any low-level manual implementations.

In these next sections we outline the overall structure of PinnDE, providing the current extent of this packages' capabilities. We then show how the process of solving a system of differential equations can be done using PinnDE.

3.1 Overview

Figure 1 shows the data flow of main modules involved in solving a system of differential equations using PinnDE. Each individual module gives the user a small amount of requirements to specify,

all of which are related to the problem specification itself as well as to the basic neural network architecture and optimization procedure, but as will be demonstrated below still does not take away from the users' ability to interact with all further parts of the solution process.

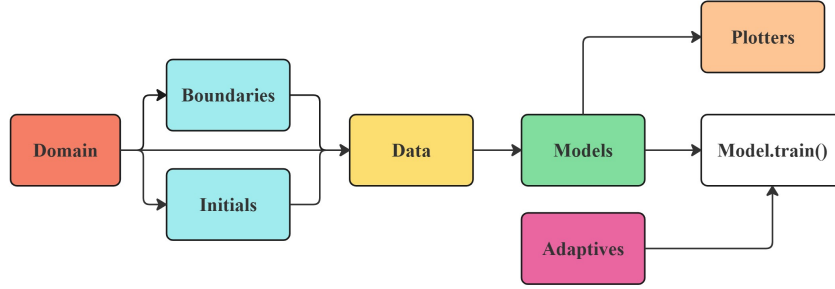


Figure 1: Flow chart of the module structure of PinnDE

A user first initializes the domain of their problem using a single function from the **Domain** module. Next, the boundary conditions of the problem are initialized with the **Boundaries** module. If the problem is time-dependent and involves initial conditions, the initial conditions are defined using a single function from the **Initials** module. Then a single function in the **Data** module is being called corresponding to the system of differential equations one wishes to solve. This function call creates a data object, which is what is passed to a function call in the **Model** module to create the desired model to solve the specified differential equations. As a next step one calls the `train()` on one's model, invoking the training routine for the specific problem with desired functionality (hard constraining, adaptive collocation points, meta-learned adaptivity, etc). Lastly, PinnDE then provides a plotting module to quickly plot losses and visualization of results on specific dimension equations.

In these next sections we give a brief overview of the domains, boundaries, data, which models and what adaptive point sampling strategies are currently available in PinnDE, specifying limitations in implementation if applicable. We only outline the general ability of PinnDE, a full API and tutorials can be found at <https://pinnde.readthedocs.io/en/latest/>.

3.1.1 Domains

The domain module is the first which must be initialized when using PinnDE. Currently we provide two general types of domains implemented, d dimensional hyper-rectangles and d dimensional ellipsoids. Both of these domains allow specifying a domain in d dimensions, denoted in PinnDE as $x_1, \dots, x_d$. Both of these have a spatial domain function, as well as a temporal domain counterpart, which is used when defining a time dependent system of partial differential equations, which allows the time dimension, denoted t in PinnDE, to be used. This enables solving $(1 + d)$ -dimensional evolution equations or d -dimensional boundary value problems. We also provide a general API for boundary functions with the abstract `boundaries` class, which when implemented allow users to define custom-domains which will work with the PinnDE architecture.

3.1.2 Initial and Boundaries

The **Boundaries** module provides boundary generation functions for ordinary and partial differential equations. The boundaries currently available to be used for partial differential equations are *Periodic*, *Dirichlet*, and *Neumann*. We further provide the boundary *ODEicbc*, which is designed to be used to specify initial or boundary conditions for systems of ordinary differential equations. All boundary functions require a domain to be passed in for which the boundary is being defined. These boundaries can be specified for any $(1 + d)$ -dimensional system of evolution equations or d -dimensional boundary value problems. Specific implementation of the boundary values depends on the domain being described.

PinnDE also offers both soft and hard constraining for a selection of these boundaries in specific dimensions. The availability of what PinnDE provides is further described in Section 3.1.4 where the **Models** of PinnDE are described. Here we give some examples of specific implementations of how hard constraining of a boundary (temporal or spatial) is done in PinnDE.

Periodic boundary conditions over n spatial dimensions $(x_1, \dots, x_n)$ of a differential equation over the hyper-rectangle $(x_1, \dots, x_n) \in [x_{l_1}, x_{r_1}] \times \dots \times [x_{l_n}, x_{r_n}]$ are enforced using a coordinate transform layer in the neural network itself, by using the transform $f(x_i) = (\cos 2\pi x_i / (x_{r_i} - x_{l_i}), \sin 2\pi x_i / (x_{r_i} - x_{l_i}))$ applied to each spatial input x_i . See [5] for further details.

For constraining a periodic boundary condition of a $(1 + 1)$ -dimensional PDE in variables (t, x) over a spatio-temporal domain $[t_0, t_f] \times \Omega$, $\Omega = [x_l, x_r]$, $x_l, x_r \in \mathbb{R}$, we only have to hard-constrain the initial conditions as periodic boundaries can be enforced exactly using a coordinate transform layer described above. For hard constraining the initial condition itself, we assume that the equation is of second order in t . Then, we can include the initial condition as a hard constraint in the neural network surrogate solution $u^\theta(t, x)$ using the ansatz

$$u^\theta(t, x) = u_0(x) + \left. \frac{\partial u}{\partial t} \right|_{t=t_0}(x) (t - t_0) + \mathcal{N}^\theta(t, x) \left(\frac{t - t_0}{t_f - t_0} \right)^2,$$

Where u_0 and $\left. \frac{\partial u}{\partial t} \right|_{t=t_0}$ represent the initial conditions at $t = t_0$, and $\mathcal{N}^\theta(t, x)$ is the neural network output. Multiple variations of possible hard constraining formulas for this problem can be created which enforce the desired initial conditions. We found this particular hard constraint ansatz to be effective for many problems we have considered, and it is therefore used in PinnDE as a default hard constraint. However more hard constraints are available in PinnDE, including custom hard constraints.

We also present the hard constraint for a two-dimensional Dirichlet boundary-value problem for a PDE in variables (x, y) over the rectangular domain $\Omega = [x_l, x_r] \times [y_l, y_u]$, with $x_l, x_r, y_l, y_u \in \mathbb{R}$. The approximate solution $u^\theta(x, y)$ is given by the equation

$$u^\theta(t, x) = A(x, y) + x^*(1 - x^*)y^*(1 - y^*)\mathcal{N}^\theta(x, y),$$

where we have

$$A(x, y) = (1 - x^*)f_{x_l}(y) + x^*f_{x_r}(y) + (1 - y^*)[g_{y_l}(x) - [(1 - x^*)g_{y_l}(x_l) + x^*g_{y_l}(x_r)]] \\ + y^*[g_{y_u}(x) - [(1 - x^*)g_{y_u}(x_l) + x^*g_{y_u}(x_r)]],$$

where $f_{x_l}, f_{x_r}, g_{y_l}, g_{y_u}$ represent some boundary functions $f(x_l, y), f(x_r, y), g(x, y_l), g(x, y_u)$ respectively, and

$$x^* = \frac{x - x_l}{x_r - x_l}, \quad y^* = \frac{y - y_l}{y_u - y_l}.$$

This formula enforces the specific boundary conditions for all output values of the neural network. This is based on what is described in [20] and is the default hard constraint for two-dimensional spatial problems over a rectangular domain in PinnDE. Other alternative hard constraints do exist in PinnDE if applicable to the scenario, and the ability for user's to create custom hard constraints also exist.

Hard constraints can be designed for general domains, however the particular domain features impact the hard constraining function, making them cumbersome to design. In PinnDE, we give designed hard constraints pre-made for the most popular use cases of domain-boundary combinations. We currently provide built-in hard constraints for rectangular domains. We provide hard constraints in the following cases: Single ODEs, single $(1 + 1)$ -dimensional PDEs with periodic boundaries, and single 2-dimensional PDEs with periodic and Dirichlet boundaries. However, PinnDE is built for custom hard constraints to be able to be created and added to the network, allowing for any user-specific hard constraint to be implemented on the network. In doing this, we cover functionality for most use cases while still having the possibility for customization for any problem.

3.1.3 Data

The **Data** module of PinnDE combines and creates sampled collocation points for training. Each model has its own specific data object, with a time data counterpart to incorporate a time dimension when training for spatio-temporal equations. The data module always takes the **Domain** and **Boundaries** as arguments. If using a time domain, it also must take in an **Initials**. Then model specific information may be passed depending on the specific data being created. For example, DeepONet data objects need the number of sensor points to sample, and inverse PINN data objects need the sampled data over the problem domain and all functions being solved for. This is then the final step before defining the model.

3.1.4 Models

PinnDE currently provides the ability to use PINNs and DeepONets for solving systems of differential equations. For PINNs, we provide standard forward PINNs as well as inverse PINNs for solving inverse problems. The number of layers and nodes can both be selected within each model function. The outer and inner activation function used within each model can be chosen by the user as well. The user also has the choice to either soft or hard constrain the network. We employ multi-layer perceptron for all neural network solution surrogates, and implement DeepONets using the architecture proposed in [24]. Both physics-informed neural networks and physics-informed deep operator networks are trained by minimizing the composite loss function (2) to approximate the solution (or solution operator) of a system of differential equations. Derivative approximations are computed using automatic differentiation [3]. Longer time integrations using DeepONets via time-stepping is realized using the procedure of [40], which iteratively applies the learned solution operator to its own output to advance the solution for arbitrary time intervals, similar as done in standard numerical methods.

3.1.5 Adaptive

When calling `train()` on a model from PinnDE, the user is allowed to pass in an adaptive point sampling strategy created from the **Adaptive**'s module. We currently provide three different adaptive sampling strategies which can be employed without needing to do any manual implementation: residual-based adaptive refinement, residual-based adaptive distribution, and residual-based adaptive refinement with distribution (RAR, RAD, RAR-D). These strategies are based on the research reported in [25, 43], and were shortly summarized in Section 2.4. These strategies can be used with both forward and inverse PINNs, and can also be applied in combination with L-BFGS optimization.

3.2 Usage

In this section we give the simple outline for how to use PinnDE. We describe what data must be declared when solving a system of differential equations. We give an example of the standard process solving a single partial differential equation. The process outline is described in Table 1.

Table 1: Workflow to solve PDE

Step 1	Use the Domain module to define the domain for the independent variables
Step 2	Define the boundary functions as Python <code>lambda</code> functions, as well as number of sample points to use on boundary
Step 3	Use the Boundaries module to set up the desired boundary
Step 4	Define all initial conditions as Python <code>lambda</code> functions, number of initial value points, and use the Initials module to set up initial conditions
Step 5	Define the Data object corresponding to the model being used
Step 6	Optionally declare number of layers of network, nodes of network, and constraint of network to interface with model
Step 7	Define the equation to solve, and use Model module and input data to solve equation
Step 8	Call <code>train(epochs)</code> on returned model to train the network
Step 9	Use trained model to plot data with given functions or take data from class to use independently

Example code that follows this structure is given in Appendix A, where the code for the following examples in Section 4 is presented. Many more tutorials are given along with the API in the linked documentation above.

4 Examples

In this section we present a few short examples of how PinnDE can be used to solve systems of ordinary differential equations, and systems of partial differential equations using the different methods implemented in PinnDE. We provide a concise description of each problem and a comparison of each trained model against the analytical solution or a numerical reference solution obtained using a standard numerical integrator using the mean squared error as comparison metric. All code for these examples is provided in Appendix A.

4.1 Rigid body

We first consider the Euler equations for the free rigid body [18], a system of ordinary differential equations, with a physics-informed network which uses soft constrained initial conditions. Specifically, we solve the system of order ordinary differential equations in angular momentum representation

$$\frac{dw_1}{dt} = \frac{I_2 - I_3}{I_2 I_3} w_2 w_3, \quad (10a)$$

$$\frac{dw_2}{dt} = \frac{I_3 - I_1}{I_1 I_3} w_1 w_3, \quad (10b)$$

$$\frac{dw_3}{dt} = \frac{I_1 - I_2}{I_1 I_2} w_1 w_2, \quad (10c)$$

where $w = (w_1, w_2, w_3)$ is the angular momentum of the rigid body, and I_1, I_2, I_3 are the principle moments of inertia. For this benchmark we use $I_1 = 0.2, I_2 = 0.3, I_3 = 0.4$, and the initial condition $w = (1, 1, 1)$. We solve this initial value problem over the interval $t \in [0, 2.5]$.

Figure 2 presents the neural network solution against a numerical solution as well as the time series of the loss for training the network. The standard numerical reference solution for this problem was computed using the standard fourth-order Runge–Kutta method. The specific architecture used was a

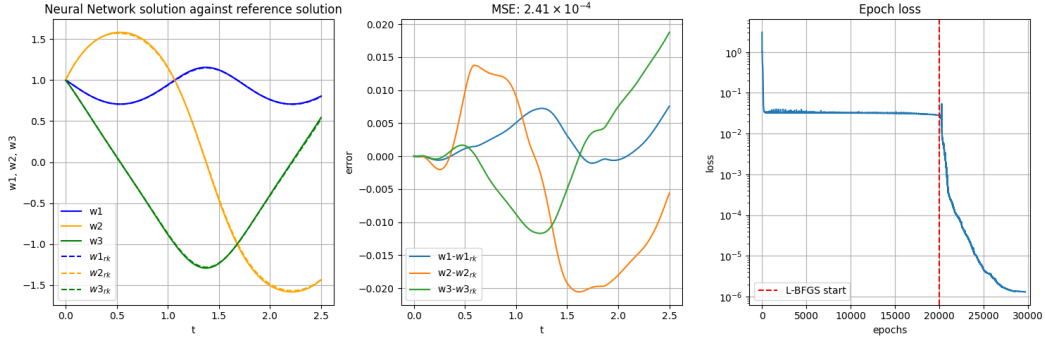


Figure 2: Solution predicted using a PINN against the numerical solution for the rigid body system (10) (left), the difference of neural network solution and numerical solution (middle), and the time series of loss from training the network (right).

PINN with 4 hidden layers with 60 nodes per layer, using the hyperbolic tangent as activation function, and Adam and L-BFGS for fine-tuning was chosen as the optimizers. We use 3000 collocation points across the domain for the network to learn the solution, sampled with Latin hypercube sampling. We train this network for 20000 epochs using Adam, then 5000 iterations using L-BFGS, implementing a two-step training routine. The start of L-BFGS optimization during training is highlighted in Fig. 2. We stop training when the epoch loss converges towards the solution. For the following examples, we will also train until a minimum loss has been reached.

The code for solving this system of equations can be found in Appendix A.1.

4.2 Lorenz model

We next use PinnDE to solve the Lorenz–1960 model, a system of ordinary differential equations, with a deep operator network which uses soft-constrained initial conditions. The dynamical system is

$$\frac{dx}{dt} = kl \left(\frac{1}{k^2 + l^2} - \frac{1}{k^2} \right) yz, \quad (11a)$$

$$\frac{dy}{dt} = kl \left(\frac{1}{l^2} - \frac{1}{k^2 + l^2} \right) xz, \quad (11b)$$

$$\frac{dz}{dt} = \frac{kl}{2} \left(\frac{1}{k^2} - \frac{1}{l^2} \right) xy, \quad (11c)$$

where in this benchmark we chose $k = 2, l = 1$, and we choose initial conditions $x(0) = 0.5, y(0) = 0.75, z(0) = 1$. We solve this initial value problem over the interval $t \in [0, 1]$.

Figure 3 presents the neural network solution against a numerical solution as well as the time series of the loss for training the network. The standard numerical reference solution for this problem was computed using the Runge–Kutta method. The specific architecture used was a DeepONet with both

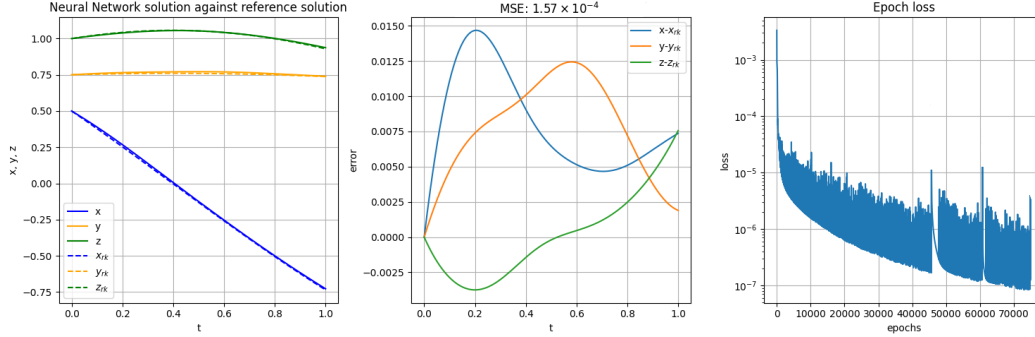


Figure 3: Solution predicted using a DeepONet against the numerical solution for the Lorenz system (11) (left), the difference of neural network solution and numerical solution (middle), and the time series of loss from training the network (right).

branch net and trunk net being multi-layer perceptrons having 4 hidden layers with 40 nodes per layer, using the hyperbolic tangent as activation function, and Adam as the chosen optimizer. We use 3000 collocation points across the domain for the network to learn the solution, sampled with Latin hypercube sampling. We sample 5000 different initial conditions for the DeepONet’s branch network in the range of $x(0), y(0), z(0) \in [-2, 2]$. We train this network for 75000 epochs using Adam. This is when the epoch loss converges towards its minimum.

We further demonstrate the time-stepping ability using a DeepONet in PinnDE in Fig. 4. We time-step the trained DeepONet for 6 steps, increasing the temporal domain to $[0, 6]$, and give the error of the neural network solution against the corresponding Runge–Kutta numerical solution. The learned neural network solution operator demonstrates the ability to time-step with a consistently low error.

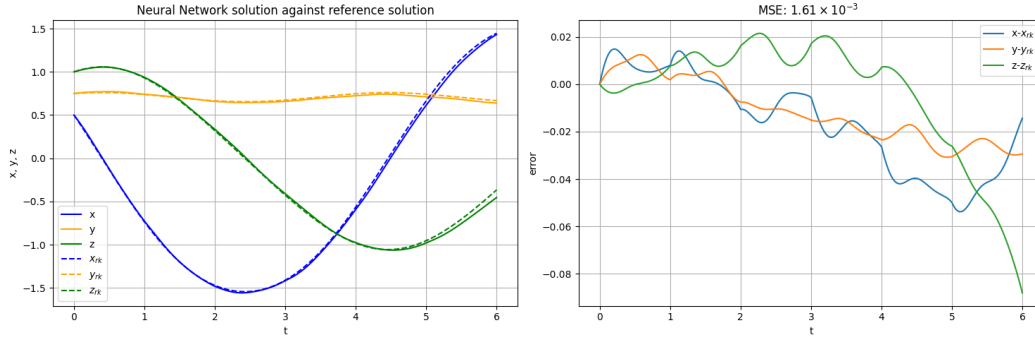


Figure 4: Time-stepped solution predicted by the trained DeepONet against the numerical reference solution obtained using a fourth-order Runge–Kutta method for system (11) (left), the difference of neural network solution and numerical solution (right).

The code for solving this system of equations can be found in Appendix A.2.

4.3 Dam break problem

We next use PinnDE to solve the classical one-dimensional dam break problem with flat bottom topography for the shallow-water equations. We use soft-constrained initial conditions and Dirichlet

boundary conditions. Specifically, we solve the one-dimensional shallow water equations given by

$$\frac{\partial h}{\partial t} + \frac{\partial(hu)}{\partial x} = 0, \quad (12a)$$

$$\frac{\partial(hu)}{\partial t} + \frac{\partial}{\partial x} \left(hu^2 + \frac{1}{2}gh^2 \right) = 0, \quad (12b)$$

where $h(t, x)$ represents the fluid depth and $u(t, x)$ represents the fluid velocity. The analytical solution for this problem is given by

$$gh_a(t, x) = \begin{cases} a_0^2 & \text{if } x - x_0 < -a_0 t \\ \frac{1}{9}(2a_0 - (x - x_0)/t)^2 & \text{if } -a_0 t < x - x_0 < 2a_0 t \\ 0 & \text{if } x - x_0 > 2a_0 t \end{cases}$$

$$u_a(t, x) = \begin{cases} 0 & \text{if } x - x_0 < -a_0 t \\ \frac{2}{3}(a_0 + (x - x_0)/t) & \text{if } -a_0 t < x - x_0 < 2a_0 t \\ 0 & \text{if } x - x_0 > 2a_0 t \end{cases}$$

where $a_0 = \sqrt{gh_0}$, see [6] for further details. This analytical solution gives the initial and boundary conditions. In this benchmark, we use initial parameters $h_0 = 0.75$, $x_0 = -0.25$, and we set $g = 1$. We consider this benchmark over the spatio-temporal domain $(t, x) \in [0.01, 1] \times [-\pi, \pi]$. We compare the neural network solution to the analytical solution for both $h(t, x)$ and $h(t, x)u(t, x)$ in Figure 5.

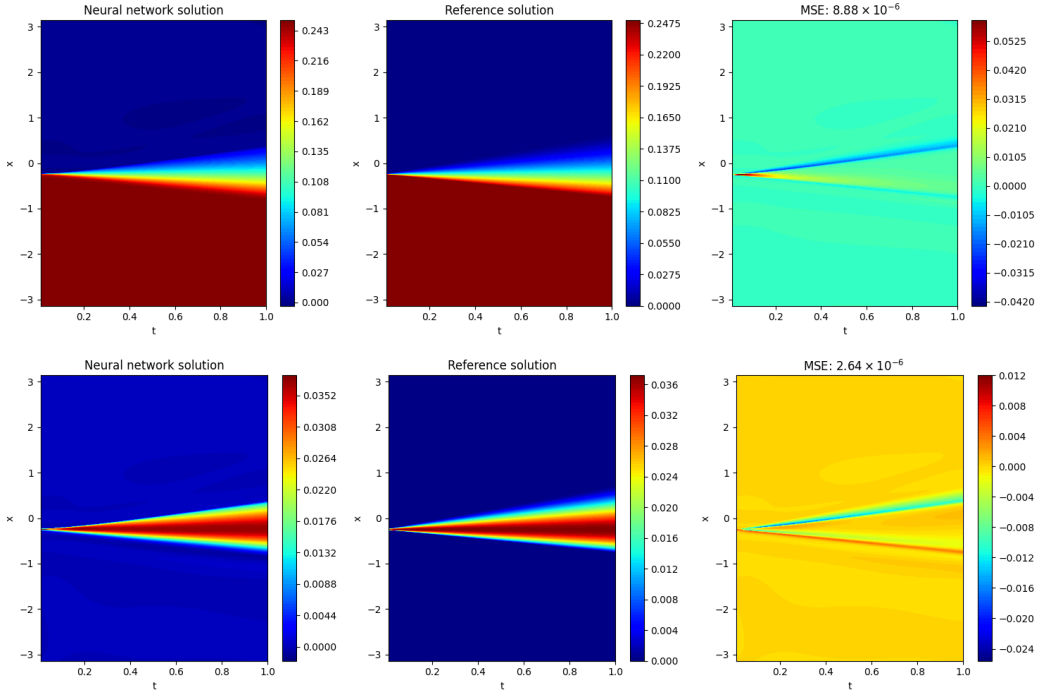


Figure 5: Solution predictions using a PINN for the dam break problem (12) (left), the analytical solution (middle), and the difference between them (right). Top: Water height $h(t, x)$. Bottom: Momentum $h(t, x)u(t, x)$.

For this example we used a PINN with 4 hidden layers with 60 nodes per layer, the hyperbolic tangent activation function, and trained the neural network using the Adam optimizer. A total of 1000 initial value collocation points were used for the network to learn the initial condition, 2000 boundary collocation points, and 15000 collocation points were used across the spatio-temporal domain for the network to learn the solution, both sampled using Latin hypercube sampling. The minimum of the loss was reached after 35000 epochs of training.

The code for solving this equation can be found in Appendix A.3.

4.4 Burgers' equation

We next use PinnDE to solve Burgers' equation with a PINN which uses soft-constrained initial and boundary conditions. We solve this equation using periodic boundary values. We also use a residual-based adaptive distribution (RAD) adaptive sampling strategy. In particular, we consider

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} - \nu \frac{\partial^2 u}{\partial x^2} = 0, \quad \nu = \frac{0.01}{\pi}. \quad (13)$$

We solve this equation over the square domain $(t, x) \in [0, 1] \times [-1, 1]$. We compare the neural network solution to a numerical reference solution in Figure 6. The numerical reference solution uses a spectral discretization in space combined with the method of lines based on the Runge–Kutta integrator for time-stepping. For this example we used a PINN with 4 hidden layers with 60 nodes

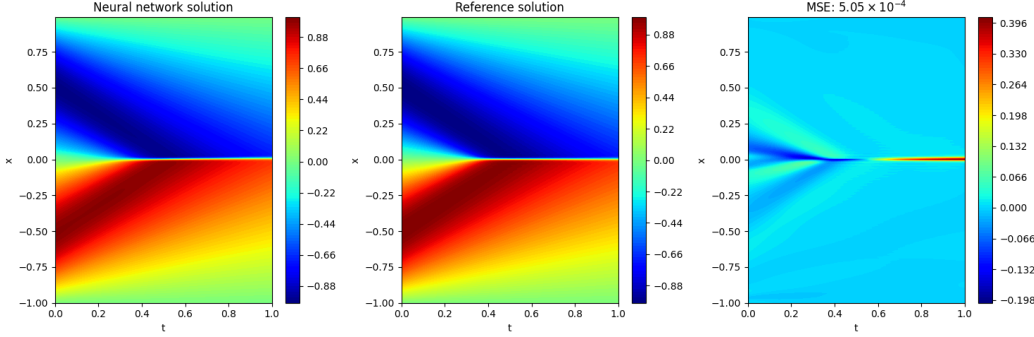


Figure 6: Solution predicted using a PINN for Burgers' equation (13) (*left*), the numerical reference solution (*middle*), and the mean squared error between them (*right*).

per layer, the hyperbolic tangent activation function, and trained the neural network using a two step training routine. The Adam optimizer was used to train the network for 5000 epochs. A total of 400 initial value collocation points were used for the network to learn the initial condition, and 2000 collocation points were used across the spatial domain for the network to learn the solution, all sampled using Latin hypercube sampling.

We used residual-based adaptive distribution (RAD) to train this equation. This strategy resamples the distribution of collocation points during training to align with areas of larger PDE residual. For Burgers' equation, this is particularly effective as we have a steep developing shock which to resolve requires locally more collocation points. We resample the distribution after 3000 epochs of training. We sample 10000 collocation points across the domain, and sample 2000 new collocation points from the probability distribution function described in Section 2.4, with hyperparameters $k = 3, c = 0$. We compare these points in Fig. 7.

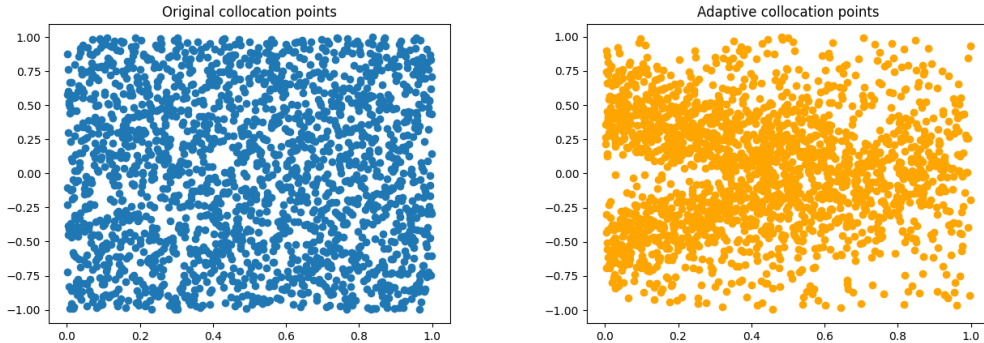


Figure 7: Original collocation points sampled using Latin hypercube sampling (*right*) and the resampled distribution by RAD (*left*).

The code for solving this equation can be found in Appendix A.4.

4.5 Heat Equation

For our next example we use PinnDE to solve the $(1 + 2)$ -dimensional linear heat equation with a DeepONet which uses soft-constrained initial and boundary conditions, meaning we use a composite loss function consisting of the PDE, initial condition, and boundary condition losses. We solve this equation with Dirichlet boundary values. Specifically, we consider the equation

$$\frac{\partial u}{\partial t} = \nu \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right), \quad \nu = 0.15, \quad (14a)$$

with initial condition

$$u_0(x, y) = \sin \pi x \sin \pi y, \quad (14b)$$

and with Dirichlet boundary conditions

$$u(t, x, y) = 0 \quad \forall (t, x, y) \in \{[0, 1] \times \mathbb{R}^2 | x^2 + y^2 = 1\} \quad (14c)$$

We solve this equation over the time interval $t \in [0, 1]$, on the spatial domain of the unit circle $(x, y) \in \{\mathbb{R}^2 | x^2 + y^2 < 1\}$. We compare the neural network solution to the analytical solution $u(t, x, y) = \exp(\nu\pi^2 t) \sin \pi x \sin \pi y$ in Figure 8.

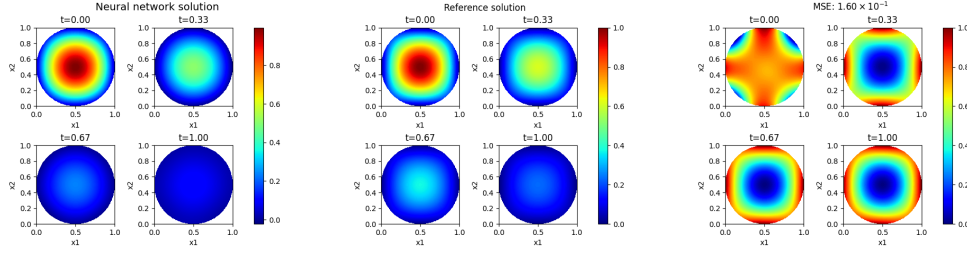


Figure 8: Solution predicted by DeepONet for the heat equation (14) (left), the analytical solution (middle), and the mean squared error between them (right).

For this example we trained a DeepONet with both branch net and trunk net being multi-layer perceptrons having 4 hidden layers with 60 nodes per layer, using the hyperbolic tangent activation function. We use Adam as our chosen optimizer. 600 initial value points were used to be sampled along $t = 0$ for the network to learn the initial condition. The soft-constrained boundary was sampled using 600 boundary value points for the neural network to learn the boundary conditions, and 12000 collocation points were used across the domain for the network to learn the solution itself, both sampled with Latin hypercube sampling. We sample a total of 1000 initial conditions from a class of truncated Fourier series with random coefficients to learn the solution operator. After training on this class of initial conditions, the operator network can be evaluated in a simple inference step for new initial conditions from the same initial condition class, an example of which is shown in Figure 8. We note here that different classes of initial conditions, such as Gaussian random fields [24] could be considered as well for operator learning, and will be implemented in PinnDE in the future. We train this network for 4000 epochs which is when the minimum loss is reached

The code for solving this equation can be found in Appendix A.5.

4.6 Inverse linear advection speed

We lastly use PinnDE to solve for an unknown advection speed in the linear advection equation with an inverse PINN which uses a soft-constrained initial condition. We use periodic boundary conditions, which in PinnDE are implemented at the neural network level as a hard constraint to impose periodic boundaries on the independent spatial variable x . We also use a residual-based adaptive refinement with distribution (RAR-D) adaptive sampling strategy. The linear advection with variable advection speed for our reference solution data is $c = \sin(\pi x)$, and the equation is described as

$$\frac{\partial u}{\partial t} + c \frac{\partial u}{\partial x} = 0, \quad (15a)$$

where we use the initial condition

$$u_0(x) = \cos \pi x. \quad (15b)$$

We solve this equation over the interval $t \in [0, 1]$, on the spatial domain $x \in [-1, 1]$. We compare the neural network solution to a numerical calculation of the solution in Figure 9, showing the success of the trained model, as it gives small point-wise and overall mean-squared errors, respectively. We use a numerical solution here as no analytical solution for this equation exists.

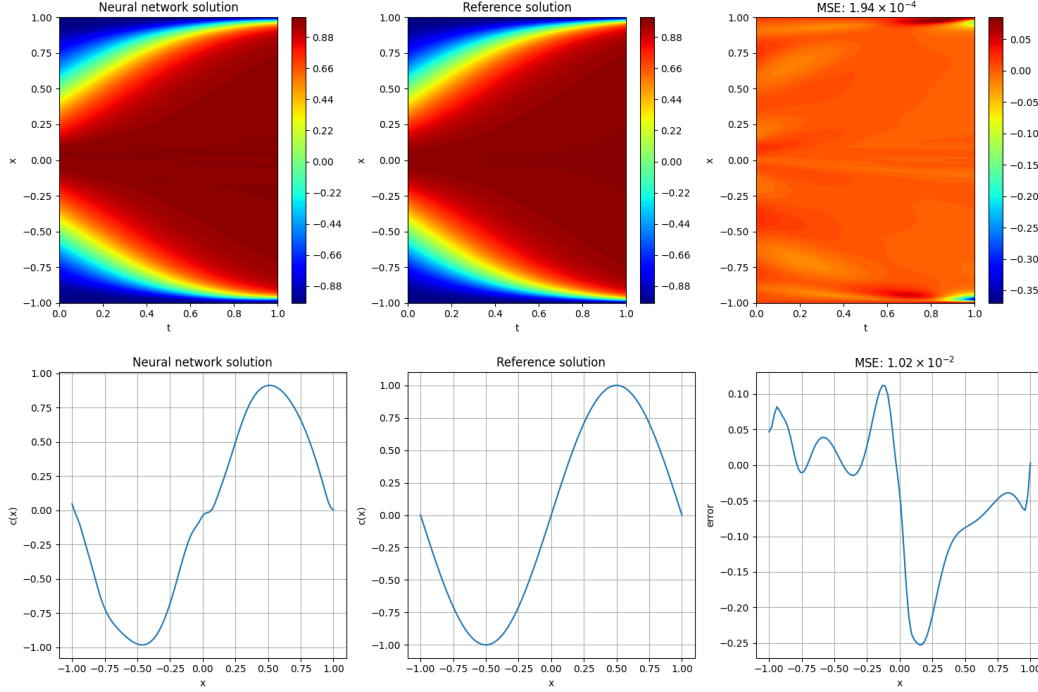


Figure 9: Solution predictions using a PINN for the inverse linear advection speed problem (15) (left), the numerical solution (middle), and the difference between them (right). Top: $u(t, x)$. Bottom: Advection velocity $c = \sin(\pi x)$.

For this example we used a PINN with 4 hidden layers with 60 nodes per layer, the hyperbolic tangent activation function, and trained the neural network using the Adam optimizer. A total of 200 initial value collocation points were used for the network to learn the initial condition, and 1000 collocation points were used across the spatio-temporal domain for the network to learn the solution, both sampled using Latin hypercube sampling. We also sampled 500 points across the spatio-temporal domain over which we evaluate the analytical solution u as data points. These data points are then used to learn the advection speed c . Our RAR-D sampling strategy added 30 collocation points every 250 epochs, with hyper-parameters $k = 3$, $c = 0$ (see [43] for more details). The evolution of collocation points throughout training is shown in Fig. 10. The minimum of the loss was reached after 5000 epochs of training.

The code for solving this equation can be found in Appendix A.6.

5 Conclusion

Deep learning-based approaches for solving differential equations have become an increasingly popular alternative for solving differential equations compared to methods based around numerical differentiation such as finite element methods, finite volume methods, and finite difference methods. PINNs and DeepONets are two methodologies of this deep learning-based approach. With these, we have a naturally meshless alternative to any of the classical numerical methods mentioned above. Deep learning-based approaches have the ability to use automatic differentiation to compute derivatives, without the need for any numerical discretizations to approximate derivatives. This streamlines the application of these methods across arbitrary domains, and by being able to incorporate physical

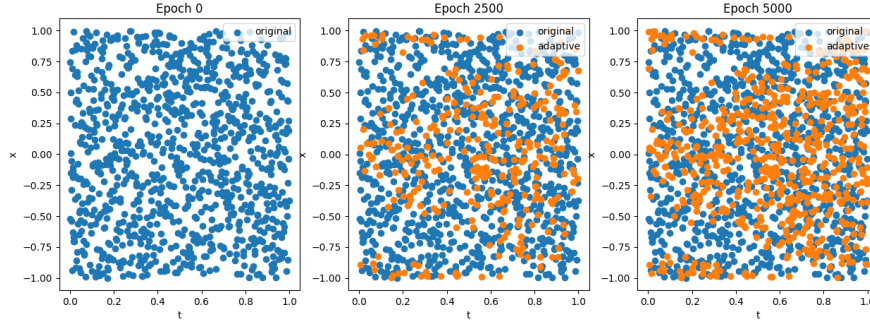


Figure 10: Original collocation points before training (*left*), the collocation points halfway through training at epoch 2500 (*middle*), and the final set of collocation points after 5000 epochs (*right*).

constraints into the solving procedure, allow for them to solve problems in which classic numerical methods may struggle.

In this paper we presented PinnDE, an open-source software package in Python for PINN and DeepONet implementations for solving ordinary and partial differential equations. We reviewed the methodologies behind physics-informed neural networks for both forward and inverse problems, the DeepONet approach to operator learning, and adaptive point sampling strategies. We then introduced PinnDE and its functionality, and finally presented several worked examples to show the effectiveness of this package.

Software packages giving the ability to use physics-informed machine learning approaches are becoming increasingly popular. In contrast to other existing packages, PinnDE stresses simplicity and requires minimal coding from the side of the user, thus alleviating the need of writing significant amounts of software to support the higher level of customization that other packages offer, while still providing many of the abilities that alternative packages offer. As such, PinnDE provides a simple interface where users with little experience can still write and understand the implementation of software written in this package. This gives researchers the ability to use this package with collaborators outside of the field who may be interested in this area of research but are lacking the skills to use more advanced packages. PinnDE also provides a tool for educators to use as the simple implementations give a low barrier of entry to new users. We also hope that by providing a package with such quick implementations of problems will promote further research and development of the field of scientific machine learning.

PinnDE is continuously being developed to integrate new advancements in research surrounding PINNs and DeepONets, while aiming to maintain a straightforward user experience. Some possible additions which will be implemented in future versions are more adaptive collocation point methods, other variants on DeepONet and PINN architectures, meta-learned optimization, the availability of different backend, and general geometries which differential equations can be solved over.

All code and link to documentation with guides can be found at: <https://github.com/JB55Matthews/PinnDE>

Acknowledgments and Disclosure of Funding

This research was undertaken thanks to funding from the Canada Research Chairs program and the NSERC Discovery Grant program.

Author Contributions

Jason Matthews: Writing - original draft, Writing - review & editing, software, conceptualization, methodology, visualization. **Alex Bihlo:** Writing - review & editing, conceptualization, methodology, resources, supervision, funding acquisition.

References

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org.
- [2] Shivam Arora, Alex Bihlo, and Francis Valiquette. Invariant physics-informed neural networks for ordinary differential equations. *Journal of Machine Learning Research*, 25:1–24, 2024.
- [3] Atilim Gunes Baydin, Barak A Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *Journal of machine learning research*, 18(153):1–43, 2018.
- [4] Alex Bihlo. Improving physics-informed neural networks with meta-learned optimization. *Journal of Machine Learning Research*, 25(14):1–26, 2024.
- [5] Alex Bihlo and Roman O Popovych. Physics-informed neural networks for the shallow-water equations on the sphere. *Journal of Computational Physics*, 456:111024, 2022.
- [6] Onno Bokhove. Flooding and drying in discontinuous Galerkin finite-element discretizations of shallow-water equations. Part 1: one dimension. *Journal of Scientific Computing*, 22(1):47–82, 2005.
- [7] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018.
- [8] Rüdiger Brecht, Elsa Cardoso-Bihlo, and Alex Bihlo. Physics-informed neural networks for tsunami inundation modeling. *Journal of Computational Physics*, page 114066, 2025.
- [9] Rüdiger Brecht, Dmytro R Popovych, Alex Bihlo, and Roman O Popovych. Improving physics-informed DeepONets with hard constraints. *arXiv preprint arXiv:2309.07899*, 2023.
- [10] Elsa Cardoso-Bihlo and Alex Bihlo. Exactly conservative physics-informed neural networks and deep operator networks for dynamical systems. *Neural Networks*, 181:106826, 2025.
- [11] Tianping Chen and Hong Chen. Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. *IEEE transactions on neural networks*, 6(4):911–917, 1995.
- [12] KR1442 Chowdhary and KR Chowdhary. Natural language processing. *Fundamentals of artificial intelligence*, pages 603–649, 2020.
- [13] Dario Coscia, Anna Ivagnes, Nicola Demo, and Gianluigi Rozza. Physics-informed neural networks for advanced modeling. *Journal of Open Source Software*, 8(87):5352, 2023.
- [14] William Falcon and The PyTorch Lightning team. Pytorch lightning, 2019.
- [15] Ling Guo, Hao Wu, and Tao Zhou. Normalizing field flows: Solving forward and inverse stochastic differential equations using physics-informed flow models. *Journal of Computational Physics*, 461:111202, 2022.
- [16] Patrik Simon Hadorn. Shift-deeponet: Extending deep operator networks for discontinuous output functions, 2022.
- [17] John M Hanna, Jose V Aguado, Sebastien Comas-Cardona, Ramzi Askri, and Domenico Borzacchiello. Residual-based adaptivity for two-phase flow simulation in porous media using physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 396:115100, 2022.
- [18] Darryl D Holm, Tanya Schmah, and Cristina Stoica. *Geometric mechanics and symmetry: from finite to infinite dimensions*, volume 12. Oxford University Press, 2009.
- [19] Jie Hou, Ying Li, and Shihui Ying. Enhancing pinns for solving pdes via adaptive collocation point movement and adaptive loss weighting. *Nonlinear Dynamics*, 111(16):15233–15261, 2023.

- [20] Isaac E Lagaris, Aristidis Likas, and Dimitrios I Fotiadis. Artificial neural networks for solving ordinary and partial differential equations. *IEEE transactions on neural networks*, 9(5):987–1000, 1998.
- [21] Isaac E Lagaris, Aristidis C Likas, and Dimitris G Papageorgiou. Neural-network methods for boundary value problems with irregular boundaries. *IEEE Transactions on Neural Networks*, 11(5):1041–1049, 2000.
- [22] Jae Yong Lee, Sung Woong Cho, and Hyung Ju Hwang. HyperDeepONet: learning operator with complex target function space using the limited resources via hypernetwork. *arXiv preprint arXiv:2312.15949*, 2023.
- [23] Xu Liu, Xiaoya Zhang, Wei Peng, Weien Zhou, and Wen Yao. A novel meta-learning initialization method for physics-informed neural networks. *Neural Computing and Applications*, 34(17):14511–14534, 2022.
- [24] Lu Lu, Pengzhan Jin, and George Em Karniadakis. Deeponet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators. *arXiv preprint arXiv:1910.03193*, 2019.
- [25] Lu Lu, Xuhui Meng, Zhiping Mao, and George Em Karniadakis. DeepXDE: A deep learning library for solving differential equations. *SIAM Review*, 63(1):208–228, 2021.
- [26] Lu Lu, Raphael Pestourie, Wenjie Yao, Zhicheng Wang, Francesc Verdugo, and Steven G Johnson. Physics-informed neural networks with hard constraints for inverse design. *SIAM Journal on Scientific Computing*, 43(6):B1105–B1132, 2021.
- [27] Levi D McClenny and Ulisses M Braga-Neto. Self-adaptive physics-informed neural networks. *Journal of Computational Physics*, 474:111722, 2023.
- [28] Michael D McKay, Richard J Beckman, and William J Conover. A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics*, 42(1):55–61, 2000.
- [29] Mohammad Amin Nabian, Rini Jasmine Gladstone, and Hadi Meidani. Efficient training of physics-informed neural networks via importance sampling. *Computer-Aided Civil and Infrastructure Engineering*, 36(8):962–977, 2021.
- [30] Guofei Pang, Lu Lu, and George Em Karniadakis. fPINNs: Fractional physics-informed neural networks. *SIAM Journal on Scientific Computing*, 41(4):A2603–A2626, 2019.
- [31] Apostolos F Psaros, Kenji Kawaguchi, and George Em Karniadakis. Meta-learning PINN loss functions. *Journal of computational physics*, 458:111121, 2022.
- [32] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378:686–707, 2019.
- [33] Maziar Raissi, Alireza Yazdani, and George Em Karniadakis. Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations. *Science*, 367(6481):1026–1030, 2020.
- [34] Carl Runge. Über die numerische Auflösung von Differentialgleichungen. *Mathematische Annalen*, 46:167–178, 1895.
- [35] Jacob Seidman, Georgios Kissas, Paris Perdikaris, and George J Pappas. NOMAD: Nonlinear manifold decoders for operator learning. *Advances in Neural Information Processing Systems*, 35:5601–5613, 2022.
- [36] Sivalingam SM, Pushpendra Kumar, and V Govindaraj. A novel optimization-based physics-informed neural network scheme for solving fractional differential equations. *Engineering with Computers*, 40(2):855–865, 2024.
- [37] Kejun Tang, Xiaoliang Wan, and Qifeng Liao. Adaptive deep density approximation for Fokker–Planck equations. *Journal of Computational Physics*, 457:111080, 2022.
- [38] Simone Venturi and Tiernan Casey. SVD perspectives for augmenting DeepONet flexibility and interpretability. *Computer Methods in Applied Mechanics and Engineering*, 403:115718, 2023.
- [39] Athanasios Voulodimos, Nikolaos Doulamis, Anastasios Doulamis, and Eftychios Protopapadakis. Deep learning for computer vision: A brief review. *Computational intelligence and neuroscience*, 2018(1):7068349, 2018.

- [40] Sifan Wang and Paris Perdikaris. Long-time integration of parametric evolution equations with physics-informed deepnets. *Journal of Computational Physics*, 475:111855, 2023.
- [41] Sifan Wang, Yujun Teng, and Paris Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5):A3055–A3081, 2021.
- [42] Colby L Wight and Jia Zhao. Solving Allen-Cahn and Cahn-Hilliard equations using the adaptive physics informed neural networks. *arXiv preprint arXiv:2007.04542*, 2020.
- [43] Chenxi Wu, Min Zhu, Qinyang Tan, Yadhu Kartha, and Lu Lu. A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 403:115671, 2023.
- [44] Liu Yang, Dongkun Zhang, and George Em Karniadakis. Physics-informed generative adversarial networks for stochastic differential equations. *SIAM Journal on Scientific Computing*, 42(1):A292–A317, 2020.
- [45] Jeremy Yu, Lu Lu, Xuhui Meng, and George Em Karniadakis. Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems. *Computer Methods in Applied Mechanics and Engineering*, 393:114823, 2022.
- [46] Lei Yuan, Yi-Qing Ni, Xiang-Yun Deng, and Shuo Hao. A-PINN: Auxiliary physics informed neural networks for forward and inverse problems of nonlinear integro-differential equations. *Journal of Computational Physics*, 462:111260, 2022.
- [47] Shaojie Zeng, Zong Zhang, and Qingsong Zou. Adaptive deep neural networks methods for high-dimensional partial differential equations. *Journal of Computational Physics*, 463:111232, 2022.
- [48] Etienne Zeudong, Elsa Cardoso-Bihlo, and Alex Bihlo. Low-rank adaptive physics-informed HyperDeepONets for solving differential equations. *arXiv preprint arXiv:2507.18346*, 2025.
- [49] Dongkun Zhang, Lu Lu, Ling Guo, and George Em Karniadakis. Quantifying total uncertainty in physics-informed neural networks for solving forward and inverse stochastic problems. *Journal of Computational Physics*, 397:108850, 2019.

A Appendix

Here we provide the code corresponding to the examples presented in Sections [4.1, 4.2, 4.3, 4.4, 4.5, 4.6].

A.1 Rigid body code

The following is the code being used in Section 4.1.

```
import src.pinnde as p
import numpy as np

re = p.domain.NRect(1, [0], [2.5])
cond = p.boundaries.odeicbc(tre, [[1], [1], [1]], "ic")

dat = p.data.pinndata(tre, cond, 3000, 300)

eqns = [ "(-5/6)*u2*u3 - u1x1", "(5/2)*u1*u3 - u2x1",
         "(-5/3)*u1*u2 - u3x1" ]

mymodel = p.models.pinn(dat, eqns)

mymodel.train(20000)
mymodel.train(5000, opt="lbfgs")
```

A.2 Lorenz model code

The numerical results presented in Section 4.2 are obtained from the following code.

```
import pinnde.legacy.ode_Solvers as ode_Solvers

eqn1 = "(-1.6)*x*y - ut"
eqn2 = "0.1*u*y - xt"
eqn3 = "0.75*u*x - yt"
eqns = [eqn1, eqn2, eqn3]
orders = [1, 1, 1]
inits = [[0.5], [0.75], [1]]
t_bdry = [0,1]
N_pde = 500
sensor_range = [-2, 2]
num_sensors = 3000
epochs = 75000

mymodel = ode_Solvers.solveODE_DeepONetSystem_IVP(eqns, orders,
                                                  inits, t_bdry, N_pde, sensor_range, num_sensors, epochs)

mymodel.timeStep(6)
mymodel.plot_epoch_loss()
mymodel.plot_solution_prediction()
```

A.3 Dam break problem code

To obtain the results presented in Section 4.3, the following code can be used.

```
import pinnde as p
import numpy as np
import tensorflow as tf

h_analytical = lambda t, x: tf.where(x+0.25<-0.5*t, 0.5**2,
                                     tf.where(x+0.25<1*t, (1-(x+0.25)/t)**2/(9), 0.0))

u_analytical = lambda t, x: tf.where(x+0.25<-0.5*t, 0.0,
                                     tf.where(x+0.25<1*t, 2/3*(0.5+(x+0.25)/t), 0.0))

h_init = lambda x: tf.where(x+0.25<-0.5*0.75, 0.5**2,
                           tf.where(x+0.25<1*0.75, (1-(x+0.25)/0.75)**2/(9), 0.0))

u_init = lambda x: tf.where(x+0.25<-0.5*0.75, 0.0,
                           tf.where(x+0.25<1*0.75, 2/3*(0.5+(x+0.25)/0.75), 0.0))

tre = p.domain.Time_NRect(1, [-np.pi], [np.pi], [0.01, 1])

bound = p.boundaries.dirichlet(tre, [h_analytical])

inits = p.initials.initials(tre, [[h_init], [u_init]])

dat = p.data.timepinndata(tre, bound, inits, 15000, 2000, 1000)

eqn1 = "u1*u2t + u1t*u2 + 2*u1*u2*u2x1 + u1x1*u2**2 + u1*u1x1"
eqn2 = "u1t + u1*u2x1 + u1x1*u2"
mymodel = p.models.pinn(dat, [eqn1, eqn2])
mymodel.train(35000)
```


A.4 Burgers' equation code

Here is the code which provides the results presented in Section 4.4.

```
import pinnde as p
import numpy as np
import tensorflow as tf

tre = p.domain.Time_NRect(1, [-1], [1], [0,1])

bound = p.boundaries.periodic(tre)
inits = p.initials.initials(tre, [lambda x: -np.sin(np.pi*x)])

dat = p.data.timepinndata(tre, bound, inits, 2000, 600, 400)

mymodel = p.models.pinn(dat, ["ut+u*ux1-(0.01/np.pi)*ux1x1"])
rad = p.adaptives.RAD(3000, 3, 0, 5)
mymodel.train(5000)
```

A.5 Heat equation code

The numerical results presented in Section 4.5 are obtained from the following code.

```
import pinnde as p
import numpy as np
import tensorflow as tf

center = [0.5, 0.5]
semilengths = [0.5, 0.5]
timerange = [0, 1]
dom = p.domain.Time_NEllipsoid(2, center, semilengths, timerange)

bdryfunc = lambda t, x1, x2: 0+t*0
bound = p.boundaries.dirichlet(tre, [bdryfunc])

u0func = lambda x1, x2: tf.sin(np.pi*x1)*tf.sin(np.pi*x2)
inits = p.initials.initials(dom, [u0func])

dat = p.data.timedondata(dom, bound, inits, 12000, 600, 600, 1000)

eqn = "0.15*ux1x1 + 0.15*ux2x2 - ut"
mymodel = p.models.deeponet(dat, [eqn])
mymodel.train(4000)
```

A.6 Inverse linear advection equation code

The numerical results presented in Section 4.6 are obtained from the following code.

```
import src.pinnde as p
import numpy as np
import tensorflow as tf

from scipy.integrate import odeint
from scipy.fftpack import diff as psdiff
from scipy.interpolate import RegularGridInterpolator

c = lambda x: tf.sin(np.pi*x)
u0 = lambda x: tf.cos(np.pi*x)

# Creating reference solution
```

```

t = np.linspace(0, 1, 200)
x = np.linspace(-1, 1, 200)
c_val = c(x)
def la(u, t, L):
    ux = psdiff(u, period=L)
    ut = -c_val*ux
    return ut
def solve_la(u0, t, L):
    sol = odeint(la, u0, t, args=(L,))
    return sol
ureference = solve_la(u0(x), t, 2)
uinterp = RegularGridInterpolator((t, x), ureference,
    method='linear')

N_data = 500
xdata = np.random.uniform(-1, 1, N_data)
tdata = np.random.uniform(0, 1, N_data)
data = np.column_stack([tdata, xdata])
udata = uinterp(data)

timerange = [0, 1]
tre = p.domain.Time_NRect(1, [-1], [1], timerange)

bound = p.boundaries.periodic(tre)

u0func = lambda x1: tf.cos(np.pi*x1)
inits = p.initials.initials(tre, [u0])

dat = p.data.timeinvpinndata(tre, bound, inits, [tdata, xdata],
    [udata], 1000, 10, 200)

eqn = "ut+c*ux1"
mymodel = p.models.invpinn(dat, [eqn], ["c"])

rard = p.adaptives.RARD(250, 30, k=3, c=0)
mymodel.train(5000, adapt_pt=rard)

```